

Jadavpur University

MCA -1st YEAR 2nd Semester -2024

Lab ASSIGNMENT - DATABASE MANAGEMENT SYSTEM

NAME-SOUMEN GHARUI ROLL- 002310503005

DATE-30.05.2024

Online Dairy Product Management System

General Description:

The Online Product Management System for dairy products is a sophisticated platform designed to revolutionize the way perishable dairy items are managed and delivered to customers. In addition to core features such as inventory management and delivery tracking, the system integrates a secure and reliable payment gateway to facilitate seamless transactions.

The payment gateway feature allows customers to securely complete their purchases online, enhancing convenience and trust in the system.

Problem Statement:

- **Inefficient Product Management:** Without the system, manual tracking of dairy products becomes prone to errors, leading to inaccuracies in stock levels, expiration dates, and shelf life management.
- **Delivery Delays and Inconsistencies:** Lack of optimization algorithms may result in inefficient delivery routes, leading to delays in product delivery and inconsistencies in service quality.
- **Increased Product Wastage:** Absence of real-time shelf life monitoring may lead to higher instances of product wastage as expired items may go unnoticed, impacting profitability and sustainability.
- **Poor Customer Experience:** Customers may experience inconvenience due to the absence of live tracking features, leading to uncertainty about the status of their orders and dissatisfaction with the service.
- **Manual Transaction Processing:** Manual handling of transactions increases the risk of errors and security breaches, potentially exposing sensitive customer information to unauthorized access.
- **Limited Scalability:** Manual processes limit the scalability of the business, making it challenging to expand operations and meet growing customer demands efficiently.

System Requirement & Analysis:

Non-Functional Requirements:

- **Performance:**

Response Time: The system should respond to user actions within a maximum of 3 seconds under normal load conditions.

Throughput: The system should support a minimum of 1000 simultaneous users without experiencing degradation in performance.

Scalability: The system should be capable of handling a 20% increase in user load without significant impact on performance.

- **Reliability:**

Availability: The system should have at least 99.9% uptime, excluding scheduled maintenance periods.

Fault Tolerance: The system should continue to operate seamlessly in the event of hardware or software failures, with minimal disruption to users.

Data Integrity: Data stored in the system should be accurate and consistent, with mechanisms in place to prevent data corruption or loss.

- **Security:**

Data Encryption: All sensitive data, including user credentials, payment information, and personal details, should be encrypted both in transit and at rest.

Authentication and Authorization: The system should employ robust authentication mechanisms (e.g., multi-factor authentication) to ensure that only authorized users can access sensitive functionalities.

Access Control: Role-based access control (RBAC) should be implemented to restrict access to certain features or data based on user roles and permissions.

Secure Payment Processing: The payment gateway should comply with industry standards (e.g., PCI DSS) to ensure secure handling of payment transactions and protection against fraud.

- **Usability:**

Intuitive User Interface: The user interface should be user-friendly and intuitive, allowing users to navigate the system easily and perform tasks without extensive training.

Accessibility: The system should be accessible to users with disabilities, following relevant accessibility standards (e.g., WCAG).

Multi-platform Compatibility: The system should be compatible with major web browsers and mobile devices to accommodate users accessing the platform from different devices.

- **Scalability:**

Horizontal Scalability: The system architecture should support horizontal scaling to accommodate increased user load or data volume by adding more resources or servers.

Vertical Scalability: The system should support vertical scaling to handle increased processing demands by upgrading hardware components (e.g., CPU, memory) without significant downtime.

- **Maintainability:**

Modularity: The system should be designed with modular components to facilitate easier maintenance and updates without impacting the entire system.

Documentation: Comprehensive documentation should be provided for system architecture, codebase, APIs, and deployment procedures to aid in maintenance and troubleshooting.

Logging and Monitoring: The system should incorporate logging and monitoring mechanisms to track system behaviour, detect anomalies, and facilitate debugging and performance optimization.

- **Compliance:**

Regulatory Compliance: The system should comply with relevant regulations and standards governing data privacy, security, and online transactions (e.g., GDPR, HIPAA, PSD2).

Industry Standards: The system should adhere to industry best practices and standards in software development, security, and quality assurance.

Functional Requirements:

- **User Authentication and Authorization:**

Users should be able to register and log in to the system using unique credentials (username/email and password).

The system should authenticate user credentials securely and grant appropriate access levels based on user roles (e.g., customer, delivery personnel, administrator).

- **Real-time Product Process Tracking:**

Users should be able to track the status of their orders in real-time, from placement to delivery.

The system should provide live updates on the processing stages of orders, including order confirmation, preparation, packaging, and delivery.

- **Product Management:**

Administrators should be able to add new products to the system by providing details such as product name, description, price, and expiration date (if applicable).

- **Inventory Management:**

The system should maintain accurate records of available stock for each product, including current quantity and location within the inventory.

Inventory levels should be automatically updated in real-time.

- **Online Payment and Invoice Generation:**

Customers should be able to select products for purchase and proceed to checkout.

The system should securely process online payments using various payment methods, including credit/debit cards, digital wallets, and other supported options.

- **Order Management:**

Customers should have the ability to view their order history, including past purchases and order statuses.

Administrators should be able to manage orders, including order processing, fulfilment, and cancellation if necessary.

SOFTWARE AND HARDWARE ANALYSIS AND REQUIREMENTS:

- SOFTWARE:

Any operating system for we browse interface

Database Oracle Database is used as database as it easy to maintain and retrieve records by simple queries which are in English language which are easy to understand and easy to write

- HARDWARE:

At least 1GB storage on the server side

Internet connection on both reader and server side

Data Collection:

- User Table

id	name	email	password	role	createdAt
1	Soumen Gharui	soumen@example.com	****	admin	2024-01-15 10:00:00
2	Sohini Das	Sohini@example.com	****	user	2024-01-16 11:00:00
3	Arka Das	Arka@example.com	****	user	2024-02-17 09:30:00
4	Subrata Das	subrata@example.com	****	user	2024-03-18 08:45:00

- Product Table

id	name	description	price	stock	category	ratings	images	expiry_date	created_at	admin_id
1	Milk	Fresh cow milk	1.50	100	Dairy	4.5	milk.png	2023-12-01	2023-01-15 10:00:00	1

2	Cheese	Cheddar cheese	3.00	50	Dairy	4.7	cheese.png	2024-01-10	2023-01-15 10:00:00	1
3	Yogurt	Greek yogurt	2.00	200	Dairy	4.8	yogurt.png	2023-11-20	2023-02-10 11:00:00	1
4	Butter	Salted butter	4.00	30	Dairy	4.6	butter.png	2023-10-15	2023-02-10 11:00:00	1

- Cart Table**

id	product_id	name	price	quantity
1	1	Milk	1.50	2
2	2	Cheese	3.00	1
3	3	Yogurt	2.00	3
4	4	Butter	4.00	1

- Orders Table**

id	user_id	total_price	order_status	created_at	paymentinfo	shipping_info_id
1	1	6.00	Completed	2023-01-20 12:00:00	Paid	1
2	2	3.00	Pending	2023-01-21 13:00:00	Unpaid	2

3	3	12.00	Completed	2023-02-22 14:00:00	Paid	3
4	4	4.00	Shipped	2023-03-23 15:00:00	Paid	4

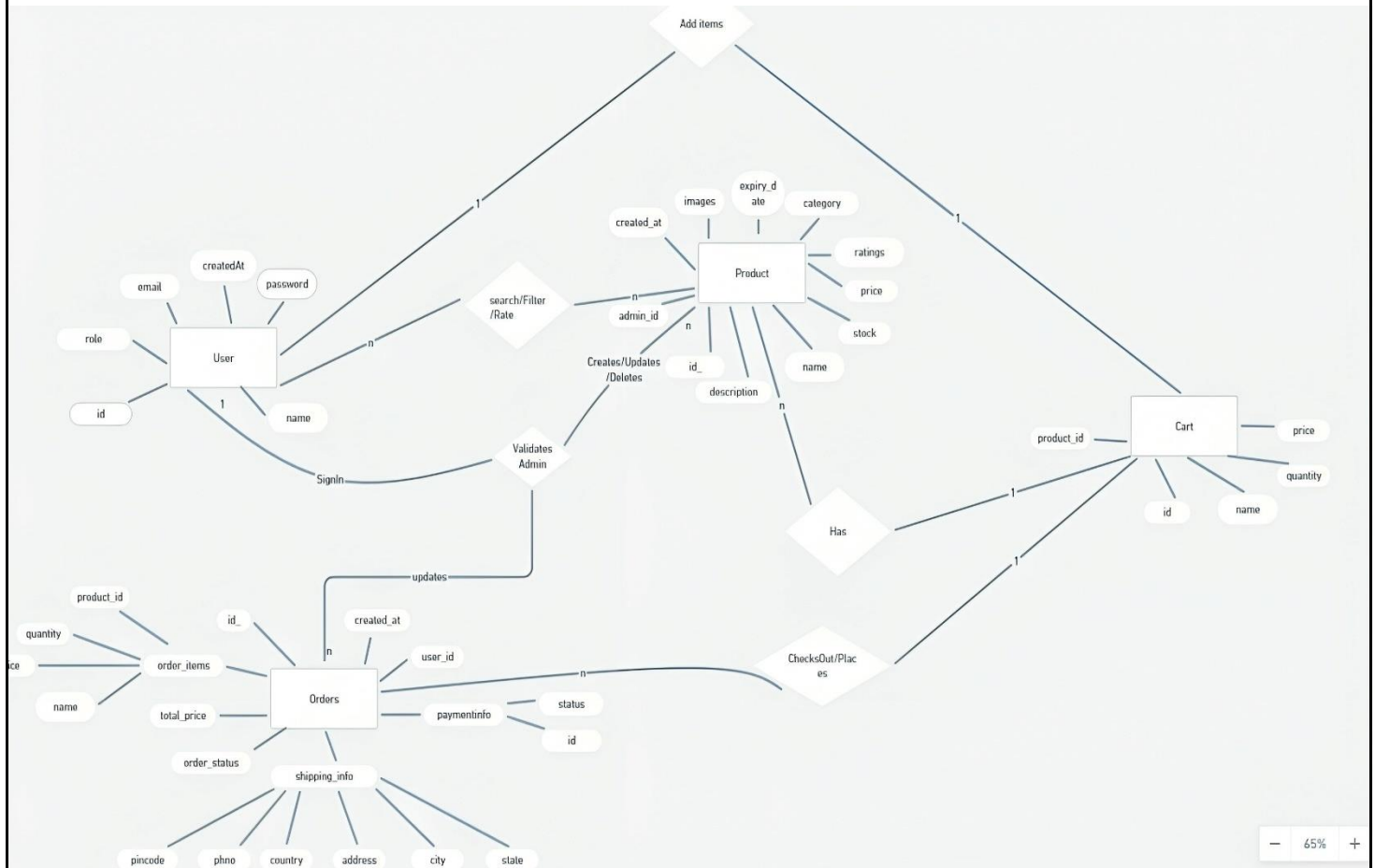
● Order Items Table

id	order_id	product_id	quantity	price
1	1	1	2	3.00
2	2	2	1	3.00
3	3	3	3	6.00
4	4	4	1	4.00

● Shipping Info Table

id	address	city	state	country	pincode	phno
1	Baghajatin	Kolkata	WB	India	12345	986446461
2	2/3 Chittaranjan Street	Kolkata	WB	India	67890	9754613566
3	B.C Road	Burdwam	WB	India	713101	963258711
4	Riksha Stand	Rishra	WB	India	762214	974125893

ER Diagram



ASSIGNMENT-01

Q1) Create the following tables:

```
SQL: CREATE TABLE EMPLOYEE (  
      EMP_CODE CHAR(16) PRIMARY KEY, EMP_NAME CHAR(20), DEPT_CODE CHAR(16), DESIG_CODE CHAR(16), SEX CHAR(1),  
      ADDRESS CHAR(25), CITY CHAR(20), STATE CHAR(20), PIN CHAR(6), BASIC NUMBER, JN_DT DATE);  
  
      CREATE TABLE DESIGNATION (DESIG_CODE CHAR(16) PRIMARY KEY, DESIG_DESC CHAR(20));  
  
      CREATE TABLE DEPARTMENT (DEPT_CODE CHAR(16) PRIMARY KEY, DEPT_NAME CHAR(20));
```

Q2) Display the structure of each table

```
SQL: desc EMPLOYEE;  
      desc DESIGNATION;  
      desc DEPARTMENT;
```

EMPLOYEE

Column	Type
EMP_CODE	CHAR(16)
EMP_NAME	CHAR(20)
DEPT_CODE	CHAR(16)
DESIG_CODE	CHAR(16)
SEX	CHAR(1)
ADDRESS	CHAR(25)
CITY	CHAR(20)
STATE	CHAR(20)
PIN	CHAR(6)
BASIC	NUMBER
JN_DT	DATE

DESIGNATION

Column	Type
DESIG_CODE	CHAR(16)
DESIG_DESC	CHAR(20)

DEPARTMENT

Column	Type
DEPT_CODE	CHAR(16)
DEPT_NAME	CHAR(20)

Q3) Insert few rows in each table.

```
SQL: -- Insert into DEPARTMENT
```

```
INSERT INTO DEPARTMENT (DEPT_CODE, DEPT_NAME) VALUES ('D1', 'Personnel');  
INSERT INTO DEPARTMENT (DEPT_CODE, DEPT_NAME) VALUES ('D2', 'Production');  
INSERT INTO DEPARTMENT (DEPT_CODE, DEPT_NAME) VALUES ('D3', 'Purchase');  
INSERT INTO DEPARTMENT (DEPT_CODE, DEPT_NAME) VALUES ('D4', 'Finance');  
INSERT INTO DEPARTMENT (DEPT_CODE, DEPT_NAME) VALUES ('D5', 'Research');
```

```
-- Insert into DESIGNATION
```

```
INSERT INTO DESIGNATION (DESIG_CODE, DESIG_DESC) VALUES ('MGR', 'Manager');  
INSERT INTO DESIGNATION (DESIG_CODE, DESIG_DESC) VALUES ('EXE', 'Executive');  
INSERT INTO DESIGNATION (DESIG_CODE, DESIG_DESC) VALUES ('OFR', 'Officer');  
INSERT INTO DESIGNATION (DESIG_CODE, DESIG_DESC) VALUES ('CLK', 'Clerk');  
INSERT INTO DESIGNATION (DESIG_CODE, DESIG_DESC) VALUES ('HLP', 'Helper');
```

```
-- Insert into EMPLOYEE
```

```
INSERT INTO EMPLOYEE (EMP_CODE, EMP_NAME, DEPT_CODE, DESIG_CODE, SEX, ADDRESS, CITY, STATE, PIN, BASIC,  
      JN_DT) VALUES ('E1', 'John Doe', 'D1', 'MGR', 'M', '123 Street', 'CityA', 'StateA', '123456',  
      50000, TO_DATE('2015-01-01', 'YYYY-MM-DD'));  
INSERT INTO EMPLOYEE (EMP_CODE, EMP_NAME, DEPT_CODE, DESIG_CODE, SEX, ADDRESS, CITY, STATE, PIN, BASIC,  
      JN_DT) VALUES ('E2', 'Jane Smith', 'D2', 'EXE', 'F', '456 Avenue', 'CityB', 'StateB', '654321',  
      40000, TO_DATE('2016-02-15', 'YYYY-MM-DD'));
```

```
INSERT INTO EMPLOYEE (EMP_CODE, EMP_NAME, DEPT_CODE, DESIG_CODE, SEX, ADDRESS, CITY, STATE, PIN, BASIC,
JN_DT) VALUES ('E3', 'Alice Brown', 'D3', 'OFR', 'F', '789 Boulevard', 'CityC', 'StateC',
'111222', 35000, TO_DATE('2017-03-20', 'YYYY-MM-DD'));
```

Q4) In EMP table insert few rows without DEPT_CODE and BASIC.

```
SQL: INSERT INTO EMPLOYEE (EMP_CODE, EMP_NAME, DESIG_CODE, SEX, ADDRESS, CITY, STATE, PIN, JN_DT)
VALUES ('E4', 'Robert White', 'CLK', 'M', '1010 Road', 'CityD', 'StateD', '333444', TO_DATE('2018-04-25',
'YYYY-MM-DD'));
```

```
INSERT INTO EMPLOYEE (EMP_CODE, EMP_NAME, DESIG_CODE, SEX, ADDRESS, CITY, STATE, PIN, JN_DT) VALUES
('E5', 'Nancy Green', 'HLP', 'F', '2020 Lane', 'CityE', 'StateE', '555666', TO_DATE('2019-05-30', 'YYYY-
MM-DD'));
```

Q5) Find the rows with unassigned DEPT_CODE

```
SQL: SELECT * FROM EMPLOYEE WHERE DEPT_CODE IS NULL;
```

EMP_CODE	EMP_NAME	DEPT_CODE	DESIG_CODE	SEX	ADDRESS	CITY	STATE	PIN	BASIC	JN_DT
E4	Robert White	NULL	CLK	M	1010 Road	CityD	StateD	333444	NULL	2018-04-25
E5	Nancy Green	NULL	HLP	F	2020 Lane	CityE	StateE	555666	NULL	2019-05-30

Q6) Find the rows with BASIC equal to zero

```
SQL: SELECT * FROM EMPLOYEE WHERE BASIC = 0;
```

Q7) Find the rows with unassigned Basic

```
SQL: SELECT * FROM EMPLOYEE WHERE BASIC IS NULL;
```

EMP_CODE	EMP_NAME	DEPT_CODE	DESIG_CODE	SEX	ADDRESS	CITY	STATE	PIN	BASIC	JN_DT
E4	Robert White	NULL	CLK	M	1010 Road	CityD	StateD	333444	NULL	2018-04-25
E5	Nancy Green	NULL	HLP	F	2020 Lane	CityE	StateE	555666	NULL	2019-05-30

Q8) Find the average basic of the employees.

```
SQL: SELECT AVG(BASIC) AS Average_Basic FROM EMPLOYEE;
```

Average_Basic
41666.67

Q9) Replace the BASIC with 0 for the rows with unassigned Basic.

```
SQL: UPDATE EMPLOYEE SET BASIC = 0 WHERE BASIC IS NULL;
```

Q10) Again, find the average Basic. (Note the difference of result obtained in Q.8 & Q.10.)

```
SQL: SELECT AVG(BASIC) AS Average_Basic FROM EMPLOYEE;
```

Average_Basic
25000

Q11) Delete the rows with unassigned DEPT_CODE.

```
SQL: DELETE FROM EMPLOYEE WHERE DEPT_CODE IS NULL;
```

Q12) Say, Net pay of an employee= Basic+HRA+DA where HRA is 50% of the Basic & DA is 40% of Basic. Show the employee name & Net pay for all employees.

```
SQL: SELECT EMP_NAME, BASIC + (0.5 * BASIC) + (0.4 * BASIC) AS NET_PAY FROM EMPLOYEE;
```

EMP_NAME	NET_PAY
John Doe	95000
Jane Smith	76000
Alice Brown	66500

Q13) Show the EMP_NAME & BASIC in the ascending order of DEPT_CODE. The employee name must appear in uppercase.

SQL: SELECT UPPER(EMP_NAME) AS EMP_NAME, BASIC FROM EMPLOYEE ORDER BY DEPT_CODE ASC;

EMP_NAME	BASIC
-----	-----
JOHN DOE	50000
JANE SMITH	40000
ALICE BROWN	35000

Q14) Find the employees who have joined after 1 st January 2010.

SQL: SELECT * FROM EMPLOYEE WHERE JN_DT > TO_DATE('2010-01-01', 'YYYY-MM-DD');

DEPT_CODE	DESIG_CODE	SEX	ADDRESS	CITY	STATE	PIN	BASIC	JN_DT
-----	-----	-----	-----	-----	-----	-----	-----	-----
E1	John Doe	D1	MGR	M	123 Street	CityA	StateA	123456 50000 2015-01-01
E2	Jane Smith	D2	EXE	F	456 Avenue	CityB	StateB	654321 40000 2016-02-15
E3	Alice Brown	D3	OFR	F	789 Boulevard	CityC	StateC	111222 35000 2017-03-20

Q15) Find, how many employees have joined in the month of January?

SQL: SELECT COUNT(*) AS January_Joins FROM EMPLOYEE WHERE EXTRACT(MONTH FROM JN_DT) = 1;

January_Joins

1

Q16) Find the maximum & minimum Basic.

SQL: SELECT MAX(BASIC) AS Max_Basic, MIN(BASIC) AS Min_Basic FROM EMPLOYEE;

Max_Basic	Min_Basic
-----	-----
50000	35000

Q17) Find how many Female employees are there?

SQL: SELECT COUNT(*) AS Female_Employees FROM EMPLOYEE WHERE SEX = 'F';

Female_Employees

2

Q18) Replace CITY with existing value converted into uppercase for all rows.

SQL: UPDATE EMPLOYEE SET CITY = UPPER(CITY);

Q19) Find in how many different cities various employees are residing?

SQL: SELECT COUNT(DISTINCT CITY) AS Different_Cities FROM EMPLOYEE;

Different_Cities

3

Q20) Display the employee information in the ascending order of DEPT_CODE and with in a Department, it should be in the descending order of BASIC.

SQL: SELECT * FROM EMPLOYEE ORDER BY DEPT_CODE ASC, BASIC DESC;

EMP_CODE	EMP_NAME	DEPT_CODE	DESIG_CODE	SEX	ADDRESS	CITY	STATE	PIN	BASIC	JN_DT
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
E1	John Doe	D1	MGR	M	123 Street	CityA	StateA	123456	50000	2015-01-01
E2	Jane Smith	D2	EXE	F	456 Avenue	CityB	StateB	654321	40000	2016-02-15
E3	Alice Brown	D3	OFR	F	789 Boulevard	CityC	StateC	111222	35000	2017-03-20

ASSIGNMENT-02

Q1) From the EMP table show the minimum, maximum and average basic for each department (show dept. Code).

SQL: SELECT DEPT_CODE, MIN(BASIC) AS Min_Basic, MAX(BASIC) AS Max_Basic, AVG(BASIC) AS Avg_Basic FROM EMPLOYEE GROUP BY DEPT_CODE;

DEPT_CODE	Min_Basic	Max_Basic	Avg_Basic
D1	50000	50000	50000
D2	40000	40000	40000
D3	35000	35000	35000

Q2) Find the number of female employees in each department (show dept. Code).

SQL: SELECT DEPT_CODE, COUNT(*) AS Female_Employees FROM EMPLOYEE WHERE SEX = 'F' GROUP BY DEPT_CODE;

DEPT_CODE	Female_Employees
D2	1
D3	1

Q3) Find the city wise no. of employees for each department (show dept. Code).

SQL: SELECT DEPT_CODE, CITY, COUNT(*) AS Num_Employees FROM EMPLOYEE GROUP BY DEPT_CODE, CITY;

DEPT_CODE	CITY	Num_Employees
D1	CityA	1
D2	CityB	1
D3	CityC	1

Q4) Show the designation wise no of employees who have joined in the year 2000 in each department. The listing should appear in the ascending order of no. of employees.

SQL: SELECT COUNT(EMP_CODE) COUNT_EMPLOYEE FROM EMPLOYEE WHERE YEAR(JN_DT)=2000 GROUP BY DEPT_CODE, DESIG_CODE ORDER BY COUNT_EMPLOYEE;

Q5) Find the department code wise total basic of male employees only for the departments for which such total is more than 50,000 and the listing should appear in the descending order of total basic.

SQL: SELECT DEPT_CODE, SUM(BASIC) AS Total_Basic FROM EMPLOYEE WHERE SEX = 'M' GROUP BY DEPT_CODE HAVING SUM(BASIC) > 50000 ORDER BY Total_Basic DESC;

DEPT_CODE	Total_Basic
D1	50000

Q6) Show the employee name, Designation description and basic for all employees.

SQL: SELECT E.EMP_NAME, D.DESIG_DESC, E.BASIC FROM EMPLOYEE E JOIN DESIGNATION D ON E.DESIG_CODE = D.DESIG_CODE;

EMP_NAME	DESIG_DESC	BASIC
John Doe	Manager	50000
Jane Smith	Executive	40000
Alice Brown	Officer	35000
Robert White	Clerk	NULL
Nancy Green	Helper	NULL

Q7) Show the employee name, Designation description, Department Name & Basic for all employees.

SQL: SELECT E.EMP_NAME, D.DESIG_DESC, DP.DEPT_NAME, E.BASIC FROM EMPLOYEE E JOIN DESIGNATION D ON E.DESIG_CODE = D.DESIG_CODE JOIN DEPARTMENT DP ON E.DEPT_CODE = DP.DEPT_CODE;

EMP_NAME	DESIG_DESC	DEPT_NAME	BASIC
John Doe	Manager	Personnel	50000
Jane Smith	Executive	Production	40000
Alice Brown	Officer	Purchase	35000

Q8) Find the department Codes in which no employee works.

SQL: SELECT DEPT_CODE FROM DEPARTMENT WHERE DEPT_CODE NOT IN (SELECT DISTINCT DEPT_CODE FROM EMPLOYEE);

DEPT_CODE
D4
D5

Q9) Find the department names where at least one employee works.

```
SQL: SELECT DEPT_NAME FROM DEPARTMENT WHERE DEPT_CODE IN (SELECT DISTINCT DEPT_CODE FROM EMPLOYEE);
```

DEPT_NAME
Personnel
Production
Purchase

Q10) Find the department names where at least 10 employees work.

```
SQL: SELECT DEPT_NAME FROM DEPARTMENT D JOIN EMPLOYEE E ON D.DEPT_CODE = E.DEPT_CODE GROUP BY DEPT_NAME
HAVING COUNT(E.EMP_CODE) >= 10;
```

Q11) Find the department code in which employee with highest Basic works.

```
SQL: SELECT DEPT_CODE FROM EMPLOYEE WHERE BASIC = (SELECT MAX(BASIC) FROM EMPLOYEE);
```

DEPT_CODE
D1

Q12) Find the Designation description of the employee with highest basic.

```
SQL: SELECT D.DESIG_DESC FROM EMPLOYEE E JOIN DESIGNATION D ON E.DESIG_CODE = D.DESIG_CODE WHERE E.BASIC
= (SELECT MAX(BASIC) FROM EMPLOYEE);
```

DESIG_DESC
Manager

Q13) Find the no. of managers in each department.

```
SQL: SELECT DEPT_CODE, COUNT(*) AS Num_Managers FROM EMPLOYEE WHERE DESIG_CODE = 'MGR' GROUP BY
DEPT_CODE;
```

DEPT_CODE	Num_Managers
D1	1

Q14) Find the maximum basic from EMP table without using MAX().

```
SQL: SELECT BASIC FROM EMPLOYEE E1 WHERE BASIC >= ALL (SELECT BASIC FROM EMPLOYEE E2);
```

BASIC
50000

Q15) Find the minimum basic from EMP table without using MIN().

```
SQL: SELECT BASIC FROM EMPLOYEE E1 WHERE BASIC <= ALL (SELECT BASIC FROM EMPLOYEE E2);
```

BASIC
35000

Q16) Find the name of the department with highest total basic. Do the same for highest average basic and maximum no. of employee.

■ Query for highest total basic

```
SQL: SELECT D.DEPT_NAME FROM DEPARTMENT D JOIN EMPLOYEE E ON D.DEPT_CODE = E.DEPT_CODE GROUP BY
D.DEPT_NAME ORDER BY SUM(E.BASIC) DESC FETCH FIRST ROW ONLY;
```

BASIC	DEPT_NAME
36750.00	PRODUCTION

--- Query for highest average basic

```
SQL: SELECT D.DEPT_NAME FROM DEPARTMENT D JOIN EMPLOYEE E ON D.DEPT_CODE = E.DEPT_CODE GROUP BY
D.DEPT_NAME ORDER BY AVG(E.BASIC) DESC FETCH FIRST ROW ONLY;
```

DEPT_NAME
Personnel

---Query for maximum numbers of employees

```
SQL: SELECT D.DEPT_NAME FROM DEPARTMENT D JOIN EMPLOYEE E ON D.DEPT_CODE = E.DEPT_CODE GROUP BY
D.DEPT_NAME ORDER BY COUNT(E.EMP_CODE) DESC FETCH FIRST ROW ONLY;
```

DEPT_NAME
Production

Q17) Insert same rows into EMP table with designation code not existing in DESIGNATION table.

```
SQL: INSERT INTO EMPLOYEE (EMP_CODE, EMP_NAME, DESIG_CODE, DEPT_CODE, BASIC, JN_DT, SEX, CITY)
VALUES ('E6', 'Charlie Green', 'XYZ', 'D1', 30000, TO_DATE('2018-06-01', 'YYYY-MM-DD'), 'M', 'CityX');
```

Q18) Delete the rows from EMP table with invalid DESIG_CODE.

```
SQL: DELETE FROM EMPLOYEE WHERE DESIG_CODE NOT IN (SELECT DESIG_CODE FROM DESIGNATION);
```

Q19) Find the name of the female employees with basic greater than the average basic of their respective department.

```
SQL: SELECT E.EMP_NAME
FROM EMPLOYEE E
JOIN (SELECT DEPT_CODE, AVG(BASIC) AS AVG_BASIC FROM EMPLOYEE GROUP BY DEPT_CODE) A
ON E.DEPT_CODE = A.DEPT_CODE
WHERE E.SEX = 'F' AND E.BASIC > A.AVG_BASIC;
```

Q20) Find the number of female managers.

```
SQL: SELECT COUNT(*)
FROM EMPLOYEE
WHERE SEX = 'F' AND DESIG_CODE = 'MGR';
```

```
+-----+
| female_managers |
+-----+
|                1 |
+-----+
```

ASSIGNMENT-03

Q1) In an organization, number of departments exists. Each department has a name & unique code. Number of employees work in each department. Each employee has unique employee code. Detailed information like name, address, city, basic, date of join are also stored. In a leave register for each employee leave records are kept showing leave type (CL/EL/ML etc.), from-date and to-date. When an employee retires or resigns then all the leave information pertaining to him are also deleted. Basic salary must be within Rs.5000 to Rs.9000. A department can not be deleted if any employee record refers to it. Valid grades are A/B/C. Employee name must be in uppercase only. Default value for joining date is system date. Design & implement the tables with necessary constraints to support the scenario depicted above.

```
SQL: CREATE TABLE DEPARTMENT (
    dcode varchar(10) primary key,
    dept_name varchar(30)
);
CREATE TABLE EMPLOYEE (
    ecode varchar(10) primary key,
    dcode varchar(10) not null,
    name varchar(30) check (name = UPPER(name)),
    address varchar(40),
    city varchar(20),
    basic decimal(9,2) check (basic >= 5000 AND basic <=9000),
    grades char(1) check(grades in ('A', 'B', 'C')),
    doj datetime default CURRENT_TIMESTAMP,
    foreign key(dcode) references DEPARTMENT(dcode)
);
CREATE TABLE LEAVE_REG (
    ecode varchar(10) not null,
    leave_type char(4) check (leave_type in ('CL','EL','ML')),
    from_date date,
    to_date date,
    foreign key(ecode) references employee(ecode) on delete cascade
);
```

```
INSERT INTO DEPARTMENT VALUES (1, 'Sales');
INSERT INTO DEPARTMENT VALUES (2, 'Marketing');
INSERT INTO DEPARTMENT VALUES (3, 'Human Resources');
```

```
INSERT INTO employee (ecode, name, dcode, address, city, basic, grades)
```

```
VALUES (1, 'John Doe', 1, '123 Main Street', 'New York', 7000.00, 'A');
INSERT INTO employee (ecode, name, dcode, address, city, basic, doj, grades)
VALUES (2, 'Jane Smith', 2, '456 Elm Avenue', 'Los Angeles', 8000.00, '2023-05-28 10:30:00', 'B');
INSERT INTO employee (ecode, name, dcode, address, city, basic, grades)
VALUES (3, 'Sarah Johnson', 1, '789 Oak Lane', 'Chicago', 5000.00, 'C');
INSERT INTO employee (ecode, name, dcode, address, city, basic, doj, grades)
VALUES (4, 'Michael Brown', 2, '987 Pine Street', 'San Francisco', 7500.00, '2022-12-15 09:00:00', 'A');
INSERT INTO employee (ecode, name, dcode, address, city, basic, grades)
VALUES (5, 'Emily Wilson', 3, '654 Maple Avenue', 'Seattle', 8500.00, 'B');
INSERT INTO employee (ecode, name, dcode, address, city, basic, grades)
VALUES (6, 'Suman Ghosh', 1, '123 Park Street', 'Kolkata', 6000.00, 'A');
```

```
INSERT INTO leave_reg (ecode, leave_type, from_date, to_date)
VALUES (1, 'CL', '2023-05-28', '2023-05-29');
INSERT INTO leave_reg (ecode, leave_type, from_date, to_date)
VALUES (2, 'EL', '2023-06-01', '2023-06-05');
INSERT INTO leave_reg (ecode, leave_type, from_date, to_date)
VALUES (3, 'ML', '2023-06-10', '2023-06-20');
```

- Q3) a) create a view showing employee code, name, dcode & Basic For a particular department.
b) Try to ensure a row into the view with valid department & also with invalid ones.
c) Find the newly inserted row in the table From which view was created .
d) Try to increment basic by Rs.100/-
e) Check it in the original table.
f) Delete the view.

```
SQL: create view emp_view as select ecode,name,dcode,basic from employee where dcode=1;
update emp_view set basic=basic+100;
select * from emp_view;
```

```
+-----+-----+-----+-----+
| ecode | name       | dcode | basic  |
+-----+-----+-----+-----+
| 1     | John Doe   | 1     | 7100.00 |
| 3     | Sarah Johnson | 1     | 5100.00 |
| 6     | Suman Ghosh | 1     | 6100.00 |
+-----+-----+-----+-----+
drop view emp_view;
```

- Q4) a) create a view Showing empcode, name, deptname, basic, leave type, From date & to date.
b) Try to insert a row in the view. Check what happens?
c) Try to increment basic by Rs.100.
d) Delete the view.

```
SQL: create view emp_view as select e.ecode,dept_name,basic,leave_type,from_date,to_date from employee e
join department d on e.dcode=d.dcode join leave_reg lr on lr.ecode=e.ecode;
select * from emp_view;
```

```
+-----+-----+-----+-----+-----+-----+
| ecode | dept_name | basic  | leave_type | from_date | to_date  |
+-----+-----+-----+-----+-----+-----+
| 1     | Sales     | 7100.00 | CL         | 2023-05-28 | 2023-05-29 |
| 3     | Sales     | 5100.00 | ML         | 2023-06-10 | 2023-06-20 |
| 2     | Marketing | 8000.00 | EL         | 2023-06-01 | 2023-06-05 |
+-----+-----+-----+-----+-----+-----+
drop view emp_view;
```

- Q5) a) Create a table having empcode , Name, deptname, & basic From the existing tables along with the records of the employee who are in a particular department (say, d1) and with a basic Rs. 7000/-
b) From the existing table, add the employees with the basic salary greater than or equal to 7000/-
c) Alter the table to add a net pay column.
d) Replace net pay with 1.5* Basic.
e) Try to remove the net pay column.

```
SQL: create table emp as(
select ecode,name,dept_name,basic from employee e
join department d on e.dcode=d.dcode
where e.dcode=1 and basic>=7000
);
alter table emp
add column net_pay float8 not null;
```

```
update emp set net_pay=basic*1.5;
```

Q6) Drop all the tables that you have created.

SQL:

```
Drop table department;
```

```
Drop table emp;
```

```
Drop table employee;
```

```
Drop table leave_reg;
```

ASSIGNMENT-04

Q1) a) Create EMP table with ECODE(primary key), ENAME, DCODE, GRADE, BASIC & JN-DT as the columns.[except BASIC & JN-DT,all columns are of char type and site of Grade is 1.]

```
SQL: CREATE TABLE EMP (
    ECODE INT PRIMARY KEY,
    ENAME VARCHAR(50),
    DCODE CHAR(1),
    GRADE CHAR(1),
    BASIC FLOAT,
    JN_DT DATE );
```

b) Insert number of rows.

```
SQL: INSERT INTO EMP (ECODE, ENAME, DCODE, GRADE, BASIC, JN_DT) VALUES (1, 'John Doe', 'A', 'B', 5000.00, '2023-01-01');
```

```
INSERT INTO EMP (ECODE, ENAME, DCODE, GRADE, BASIC, JN_DT) VALUES (2, 'Jane Smith', 'B', 'C', 7000.00, '2023-02-15');
```

```
INSERT INTO EMP (ECODE, ENAME, DCODE, GRADE, BASIC, JN_DT) VALUES (3, 'Michael Johnson', 'A', 'A', 9000.00, '2023-03-10');
```

```
INSERT INTO EMP (ECODE, ENAME, DCODE, GRADE, BASIC, JN_DT) VALUES (4, 'Chompers Ike', 'B', 'A', 1900.00, '2022-12-05');
```

```
INSERT INTO EMP (ECODE, ENAME, DCODE, GRADE, BASIC, JN_DT) VALUES (5, 'Emily Johnson', 'A', 'B', 5500.00, '2023-04-20');
```

```
INSERT INTO EMP (ECODE, ENAME, DCODE, GRADE, BASIC, JN_DT) VALUES (6, 'Robert Williams', 'B', 'C', 7200.00, '2023-05-10');
```

```
INSERT INTO EMP (ECODE, ENAME, DCODE, GRADE, BASIC, JN_DT) VALUES (7, 'Sarah Davis', 'A', 'A', 8500.00, '2023-06-01');
```

Q2) change the column heading as shown below, So that in subsequent SELECT statement newly set heading will be shown:

ECODE	EMPLOYEE CODE
ENAME	NAME
DCODE	DEPT.CODE
JN-DT	JOINING DATE

```
SQL: ALTER TABLE EMP RENAME COLUMN ECODE TO EMPLOYEE_CODE;
```

```
ALTER TABLE EMP RENAME COLUMN ENAME TO NAME;
```

```
ALTER TABLE EMP RENAME COLUMN DCODE TO DEPARTMENT_CODE;
```

```
ALTER TABLE EMP RENAME COLUMN `JN_DT` TO JOINING_DATE;
```

Field	Type	Null	Key	Default	Extra
EMPLOYEE_CODE	int	NO	PRI	NULL	
NAME	varchar(50)	YES		NULL	
DEPARTMENT_CODE	char(1)	YES		NULL	
GRADE	char(1)	YES		NULL	
BASIC	float	YES		NULL	
JOINING_DATE	date	YES		NULL	

Q3) Set the format of columns as mentioned below, So that in subsequent SELECT statement ,values appear in the specified format:

*format of BASIC is such that a value of 7000 will be shown as7,000

*Format of GRADE will be such that full column name appears in the display.

*For JN-DT format is such that 01-JAN-00 will be shown as JANURY 01,2000

SQL: SELECT

```
EMPLOYEE_CODE AS 'EMPLOYEE CODE', NAME AS 'NAME',  
DEPARTMENT_CODE AS 'DEPT.CODE', CONCAT('Grade: ', GRADE) AS 'GRADE',  
FORMAT(BASIC, 0) AS 'BASIC',  
DATE_FORMAT(JOINING_DATE, '%M %d, %Y') AS 'JOINING DATE' FROM EMP;
```

EMPLOYEE_CODE	NAME	DEPT.CODE	GRADE	BASIC	JOINING DATE
1	John Doe	A	Grade: B	5,000	January 01, 2023
2	Jane Smith	B	Grade: C	7,000	February 15, 2023
3	Michael Johnson	A	Grade: A	9,000	March 10, 2023
4	Chompers Ike	B	Grade: A	1,900	December 05, 2022
5	Emily Johnson	A	Grade: B	5,500	April 20, 2023
6	Robert Williams	B	Grade: C	7,200	May 10, 2023
7	Sarah Davis	A	Grade: A	8,500	June 01, 2023

Q4) a) Show the display attributes of all the columns.

SQL: SHOW CREATE TABLE EMP;

Table	Create Table
EMP	CREATE TABLE `emp` (`EMPLOYEE_CODE` int NOT NULL, `NAME` varchar(50) DEFAULT NULL, `DEPARTMENT_CODE` char(1) DEFAULT NULL, `GRADE` char(1) DEFAULT NULL, `BASIC` float DEFAULT NULL, `JOINING_DATE` date DEFAULT NULL, PRIMARY KEY (`EMPLOYEE_CODE`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

b) Show the display attributes of particular column.

SQL: DESCRIBE EMP NAME;

Field	Type	Null	Key	Default	Extra
NAME	varchar(50)	YES		NULL	

c) Suppress the newly set attributes of JN-DT .Try a select statement.

SQL: ALTER TABLE EMP MODIFY COLUMN JOINING_DATE DATE;

d) Reset the newly set attributes of JN-DT

SQL: ALTER TABLE EMP MODIFY COLUMN JOINING_DATE DATE FORMAT 'YYYY-MM-DD';

e) Reset the newly set attributes of all columns.

```
SQL: ALTER TABLE EMP MODIFY COLUMN EMPLOYEE_CODE INT;  
ALTER TABLE EMP MODIFY COLUMN NAME VARCHAR(50);  
ALTER TABLE EMP MODIFY COLUMN DEPARTMENT_CODE CHAR(1);  
ALTER TABLE EMP MODIFY COLUMN GRADE CHAR(1);  
ALTER TABLE EMP MODIFY COLUMN BASIC FLOAT;  
ALTER TABLE EMP MODIFY COLUMN JOINING_DATE DATE;
```

f) Shown the display attributes of all columns.

SQL: DESCRIBE EMP;

Field	Type	Null	Key	Default	Extra
EMPLOYEE_CODE	int	NO	PRI	NULL	
NAME	varchar(50)	YES		NULL	
DEPARTMENT_CODE	char(1)	YES		NULL	
GRADE	char(1)	YES		NULL	
BASIC	float	YES		NULL	
JOINING_DATE	date	YES		NULL	

